

UNIVERSIDAD VERACRUZANA
FACULTAD DE ESTADÍSTICA E INFORMÁTICA
LICENCIATURA EN INGENIERÍA DE SOFTWARE
Diseño de Software

Documento de Arquitectura de Software (SAD)

Plantilla del proyecto final · versión ligera 5.0

Proyecto	Bastión
Equipo e integrantes	Equipo 4 · Ángel Gabriel Aguilar Hernández · José Eduardo Prior Hernández
Versión del documento	v0.1
Entrega	Primera entrega
Fecha	05/10/2026

Cómo usar esta plantilla

1. Llenen lo que corresponde a la entrega en curso (primera entrega o entrega final). Lo demás puede quedar vacío por ahora.
2. Documenten a fondo sólo lo que obliga a la arquitectura a tomar cierta forma. Si algo no aplica, díganlo en una línea y expliquen por qué.
3. Usen sólo siete identificadores: **QAS** (escenario de calidad), **ASR**, **ADR**, **C** (módulo o componente), **V** (vista), **EV** (evidencia) y **R** (riesgo o pendiente). Todo lo demás se nombra.
4. Cada decisión se escribe una sola vez, en su ficha de la sección 3. Las demás secciones la citan por su número de ADR.
5. Antes de empezar, lean el ejemplo resuelto del Sistema de Gestión Hotelera: muestra cada sección bien llenada.
6. Esta plantilla pide lo necesario para un trabajo logrado; lo que distingue un trabajo sobresaliente está en la rúbrica. Si algo aquí contradice el Manual o la Evaluación vigentes, mandan esos documentos. Si ya empezaron con otra versión de la plantilla, trasladen su contenido: el formato anterior no se recalifica.

La cadena que el documento debe permitir seguir: escenario, función o restricción → **ASR** → **ADR** → elemento **C** → vista **V** → evidencia **EV** → riesgo residual **R**

1. Control y glosario

Primera entrega

Mantengan esta sección breve y actualícenla en cada entrega.

1.1 Control e historial

Sistema	Bastión: juego de escritorio cliente-servidor en el que dos o cuatro jugadores mueven su peón y colocan muros por turnos para llegar primero a la orilla opuesta del tablero, con cuentas, ranking, economía cosmética y moderación.
Propósito del documento	Explicar qué obliga a la arquitectura a tomar su forma, qué decisiones se tomaron y con qué alternativas se compararon, para que el equipo construya sin romperlas y el profesor pueda evaluarlas.
Audiencia	Los dos integrantes del equipo (para construir y mantener el sistema con un criterio común) y el profesor de la experiencia educativa (para evaluar y para confirmar las decisiones abiertas).
Alcance	Cubre el cliente de escritorio, el servidor de partidas, los contratos de comunicación y la base de datos, con las funciones de los casos de uso CU-01 a CU-49. Quedan fuera el diseño gráfico y sonoro, el contenido de los catálogos (objetos, lecciones, palabras prohibidas) y la operación en producción: el sistema se despliega en las máquinas del equipo y en el aula.

Versión	Fecha	Cambios principales
v0.1	05/10/2026	Primera entrega: control, glosario, drivers (contexto, interesados, funciones clave, restricciones, escenarios de calidad y ASR), decisiones ADR, módulos iniciales, vistas V-01 y V-02, y riesgos iniciales.

1.2 Resumen de la arquitectura

Máximo una página: problema, drivers principales, estructura general, decisiones esenciales y riesgos abiertos. Conviene escribirlo al final de cada entrega.

Problema. Bastión es un juego de mesa por turnos inspirado en Quoridor, con partidas en línea de dos o cuatro jugadores, partidas contra la IA, ranking por elo, economía cosmética con monedas y cajas, chat, espectadores y moderación (CU-01 a CU-49). Lo construye un equipo de dos personas, una en macOS arm64 y otra en Windows, con un stack propio aprobado por el profesor: C# sobre .NET 10, MonoGame DesktopGL, CoreWCF y SQL Server, sin tecnologías web.

Drivers principales. Que un cliente modificado no pueda hacer trampa (QAS-01) y que las contraseñas resistan intentos en serie (QAS-02); que una jugada llegue al rival en menos de 500 ms (QAS-03); que una desconexión o una falla de la base no deje partidas ni compras a medias (QAS-05 a QAS-07); y que todo funcione igual en los dos sistemas (QAS-08). De ahí salen siete ASR: servidor como única autoridad (ASR-01), notificación del servidor al cliente (ASR-02), multiplataforma (ASR-03), seguridad sin cifrado de transporte (ASR-04), atomicidad (ASR-05), reglas aisladas y probables (ASR-06) e internacionalización (ASR-07).

Estructura general. Cliente-servidor en capas (ADR-001): el cliente MonoGame (C-01) solo conoce los contratos (C-02); el servidor (C-03) expone CoreWCF, sus servicios (C-04) validan con un dominio sin dependencias (C-05) y guardan con EF Core (C-06) en SQL Server (C-07). Las vistas V-01 y V-02 lo muestran.

Decisiones esenciales. CoreWCF dúplex sobre net.tcp con llamadas asíncronas (ADR-002); MonoGame

DesktopGL (ADR-003); `SecurityMode.None` con la seguridad en el servidor (ADR-004); EF Core y SQL Server 2022 con el esquema en scripts SQL (ADR-005); resultados codificados traducidos en el cliente (ADR-006).

Riesgos abiertos. La confirmación de EF Core por el profesor (R-01), las credenciales en claro en la red (R-02), la presentación del tablero (R-03), SQL Server bajo Rosetta (R-04), el servidor como punto único de falla (R-05) y la latencia aún sin medir (R-06).

1.3 Glosario

Término	Significado en este proyecto
Cliente	Aplicación de escritorio (MonoGame DesktopGL) que dibuja las pantallas y envía las acciones del jugador. No decide nada: solo muestra el estado que le da el servidor.
Servidor de partidas	Proceso .NET que expone los servicios CoreWCF, valida toda regla del juego y de la cuenta, y es el único que accede a la base de datos.
Contrato	Interfaz de servicio o de callback y sus tipos de datos (Bastion.Contracts), compartidos por cliente y servidor.
Callback	Llamada que el servidor hace al cliente por el canal dúplex para avisarle de un cambio (jugada del rival, mensaje de chat, fin de partida) sin que el cliente lo pida.
Dúplex net.tcp	Canal TCP binario de CoreWCF que permite llamadas en ambos sentidos sobre la misma conexión.
Token de sesión	Valor aleatorio que el servidor entrega al iniciar sesión y que acompaña cada llamada posterior. Solo se guarda su resumen SHA-256; vence a las 24 horas (CU-01 RN-07).
Partida	Juego entre dos o cuatro participantes, o contra la IA, en un <i>modo</i> . Termina por llegada a meta, reloj, rendición, tablas o abandono (CU-25 RN-01).
Participación	Registro de un jugador en una partida, con su elo de entrada y su resultado. Un jugador tiene como mucho una sin terminar (D-01, D-11).
Modo	Variante de juego: clásico 9×9 a dos jugadores con diez muros, cuatro jugadores 9×9 con cinco muros, rápida 7×7 con seis muros (CU-17 RN-02).
Muro	Pieza de dos casillas que se coloca en el surco entre casillas; nunca puede dejar a un jugador sin camino a su meta (CU-21 RN-05).
Elo	Puntuación de habilidad por modo que cambia al terminar una partida clasificatoria y ordena el ranking.
Partida clasificatoria	La que se crea por emparejamiento (CU-17). Es la única que otorga elo y monedas; las de sala privada y contra la IA no.
Sala privada	Partida que se arma con un código o una invitación (CU-18, CU-19, CU-32).
Invitado	Cuenta sin correo ni contraseña atada a un token guardado en el dispositivo; puede vincularse después a una cuenta registrada (CU-06, CU-07).
Sanción	Restricción que aplica un moderador, de ámbito cuenta o chat, temporal o permanente (D-05).
Bitácora	Registro de solo inserción de accesos y de acciones de moderación (CU-48).
Resultado codificado	Valor de enumeración que el servidor devuelve en lugar de un texto; el cliente lo traduce con sus recursos .resx.
Usuario de mínimo privilegio	Inicio de sesión de SQL Server BastionServerConnection, sin permiso DELETE, con el que el servidor accede a la base de datos.

1.4 Uso de IA generativa

Una fila por uso significativo. No anexen conversaciones.

Qué pidieron	Qué aceptaron y qué rechazaron	Cómo lo verificaron
Apoyo en investigación: comparar opciones de interfaz (MonoGame, Godot, Unity, Stride, Avalonia), de comunicación (CoreWCF frente a WCF de .NET Framework) y de base de datos que funcionaran en macOS arm64 y en Windows.	Se aceptaron como punto de partida los resúmenes de ventajas y limitaciones; se rechazó toda opción prohibida por el curso o que no corriera en ambos sistemas. La elección final de cada tecnología la tomó el equipo y está en sus ADR.	Se consultó la documentación oficial de cada tecnología y se compiló y ejecutó un proyecto mínimo en las dos máquinas del equipo antes de adoptarla.
Completación de código: sugerencias al escribir servicios, repositorios, pantallas y pruebas.	Se aceptaron las sugerencias que cumplieran el estándar de codificación y las reglas de dependencia; se rechazaron las que ponían reglas en el cliente o mezclaban capas.	Revisión del código por ambos integrantes, compilación con <code>EnforceCodeStyleInBuild</code> y pruebas unitarias con <code>xUnit</code> .
Elaboración del documento: redacción y formato en LaTeX de esta plantilla a partir de los casos de uso y de las decisiones del equipo.	Se aceptó la redacción que reflejaba lo decidido; se corrigió o eliminó lo que no correspondía a los casos de uso o al código.	Los autores revisaron y validaron cada sección contra los casos de uso, el modelo de datos y el repositorio del código.

2. Drivers

Primera entrega

Expliquen qué obliga a la arquitectura a tomar cierta forma. Un ASR puede venir de un escenario de calidad, pero también de una función, una restricción, un objetivo de negocio o una condición del equipo.

2.1 Contexto y frontera

En un párrafo: qué está dentro del sistema, qué queda fuera y con qué sistemas externos se comunica. El diagrama va en la sección 6 como V-01; aquí sólo refiéranlo.

Dentro del sistema están el **cliente de escritorio** (pantallas, entrada del jugador, idiomas en-US y es-MX), el **servidor de partidas** (cuentas, sesiones, emparejamiento, salas, partidas en curso y contra la IA, chat, espectadores, ranking, tienda y cajas, moderación y bitácoras) y la **base de datos** SQL Server, a la que solo llega el servidor. Los actores humanos son el Jugador, el Invitado, el espectador Anónimo, el Moderador y el Administrador (lista de casos de uso). Fuera del sistema quedan el **servicio de correo**, que el servidor usa para verificar cuentas, recuperar contraseñas y confirmar cambios de correo (CU-02, CU-03, CU-04, CU-09, CU-11), y el **proveedor del segundo factor** (CU-01 RN-08); ambos se usan detrás de una interfaz. No hay pagos con dinero real (CU-38 RN-01), ni navegador, ni servicios web. El diagrama de contexto es la vista V-01 (sección 6).

2.2 Interesados y preocupaciones

Incluyan sólo a quienes tienen una preocupación que puede cambiar la arquitectura. Una preocupación es lo que le importa a alguien, no una función del sistema.

Interesado	Qué le preocupa
Jugador (cuenta registrada)	Que nadie pueda hacer trampa con un cliente modificado; que una jugada se vea en el tablero del rival casi al instante; no perder una partida por un corte breve de red; no perder monedas ni objetos por una falla a mitad de una compra.
Invitado	Poder jugar sin registrarse y conservar su progreso si después vincula la cuenta (CU-06, CU-07).
Espectador (con o sin cuenta)	Ver partidas sin poder soplar jugadas ni molestar a los participantes (CU-34).
Moderador y administrador	Que las sanciones surtan efecto de inmediato y que las bitácoras no puedan alterarse ni borrarse (CU-44, CU-48).
Equipo de desarrollo (dos integrantes, uno en macOS arm64 y otro en Windows)	Compilar, ejecutar y probar todo en ambos sistemas; trabajar en paralelo sin pisarse; poder probar las reglas del servidor sin base de datos ni red.
Profesor de la experiencia educativa	Que se respeten las tecnologías permitidas por el curso (sin stack web, WebSockets, SignalR, gRPC ni REST) y que cada decisión esté justificada y trazada hasta su evidencia.

2.3 Funciones clave

Usen los casos de uso o las historias de usuario que ya tienen, con sus identificadores. Señalen cuáles obligan a tomar decisiones arquitectónicas. Las demás pueden seguir la estructura general del sistema y documentarse sólo cuando sea necesario.

Función (CU o HU)	¿Arquitectónicamente significativa?	Por qué
CU-01 · Iniciar sesión	Sí	Credenciales que viajan por un canal sin seguridad de transporte; mensaje y tiempo de respuesta iguales ante cualquier fallo (RN-01); bloqueo escalonado (RN-03); token de 24 horas que toda llamada posterior debe presentar (RN-07).
CU-02, CU-03, CU-04, CU-09 · Registro, recuperación, verificación y cambio de credenciales	Sí	Dependen de un servicio de correo externo que puede fallar o tardar; los tokens de verificación caducan y el correo vigente no cambia hasta confirmar el nuevo (D-20).
CU-17 · Buscar partida clasificatoria	Sí	Estado compartido entre muchos clientes (la cola de emparejamiento); la partida y todas sus participaciones se crean en una sola transacción (RN-10); una sola participación sin terminar por jugador (RN-01, D-01).
CU-20 · Mover el peón y CU-21 · Colocar un muro	Sí	El servidor es la única autoridad sobre el tablero (RN-01, RN-07); toda jugada lleva número de orden para no aplicarse dos veces (CU-20 RN-06); un muro nunca puede cerrar el camino de ningún jugador, lo que obliga a recorrer el tablero (CU-21 RN-05); el rival debe enterarse sin preguntar.
CU-25 · Finalizar la partida y aplicar la progresión	Sí	Elo, estadísticas, monedas y cierre de participaciones en una sola transacción (RN-12); si falla, la partida queda pendiente de cierre y se reintenta (RN-13).
CU-24 · Abandonar y CU-26 · Reconectar	Sí	Fallas de red: la desconexión no detiene el reloj, el rival puede reclamar a los 60 segundos (CU-24 RN-06) y el cliente se reconstruye solo con el estado completo del servidor (CU-26 RN-02, RN-06).
CU-27 · Jugar contra la IA	Sí	La IA juega en el servidor con las mismas reglas que el jugador (RN-04): el motor de reglas debe poder usarse sin red ni base de datos.
CU-28 · Enviar un mensaje de chat	Sí	Filtro de palabras obligatorio en el servidor; si no puede aplicarse, el mensaje se rechaza (RN-01, RN-11); ritmo de envío acotado (RN-04).
CU-34 · Ver una partida como espectador	Sí	Difusión a hasta 200 espectadores por partida (RN-03) con ocho segundos de retraso (RN-01), sin persistencia (D-19).
CU-38 · Comprar un objeto y CU-39 · Comprar y abrir una caja	Sí	Economía: el precio y el sorteo los decide el servidor (CU-38 RN-03, CU-39 RN-08); objeto, movimiento de monedas y saldo en una sola transacción (CU-38 RN-08); el cliente nunca reenvía una compra (RN-09).
CU-44 · Aplicar una sanción y CU-48 · Consultar las bitácoras	Sí	La sanción de cuenta invalida los tokens vigentes (CU-01 RN-07); las bitácoras solo admiten inserción (CU-48 RN-02).
CU-05 a CU-16, CU-18, CU-19, CU-22, CU-23, CU-29 a CU-33, CU-35 a CU-37, CU-40 a CU-43, CU-45 a CU-47, CU-49	No	Siguen la estructura general: una pantalla del cliente, una operación del contrato y un servicio con su repositorio. Reutilizan las decisiones que imponen los casos anteriores (sesión, autoridad del servidor, transacciones).

2.4 Restricciones, supuestos y condiciones del negocio

Una restricción viene impuesta: el equipo no la decide. Un supuesto es algo que creen cierto pero no han comprobado. Si una fila habla de tiempos de respuesta o de carga, probablemente es un escenario (2.5).

Tipo	Enunciado	Qué pasa si cambia o resulta falso
Restricción	El curso pide comunicación dúplex sobre net.tcp con llamadas asíncronas y prohíbe el stack web, WebSockets, SignalR, gRPC y REST; el equipo la cumple con su stack propio (CoreWCF, ADR-002).	Si se permitiera otra tecnología, la capa de comunicación cambiaría, pero los contratos y el servidor como autoridad se conservarían.
Restricción	El lenguaje es C# sobre .NET; el sistema es una aplicación de escritorio cliente-servidor.	Cambiar de lenguaje o de plataforma invalidaría todas las decisiones tecnológicas.
Restricción	El sistema debe compilar y ejecutarse en macOS arm64 y en Windows, porque cada integrante trabaja en uno de ellos.	Si ambos usaran Windows, volverían a ser viables WCF de .NET Framework y MonoGame WindowsDX, pero no cambiaría la estructura en capas.
Restricción	La base de datos es SQL Server; los datos se guardan en Unicode con la intercalación Modern_Spanish_100_CI_AS_SC_UTF8 (D-21).	Otro motor obligaría a reescribir los scripts y a revisar los índices filtrados que garantizan reglas como la de una sola participación sin terminar.
Supuesto	El profesor acepta EF Core como framework de persistencia en lugar del Entity Framework visto en clase (pendiente de confirmar, R-01).	Habría que cambiar el acceso a datos a otra opción, por ejemplo ADO.NET; los repositorios de C-06 aíslan el cambio del resto del servidor.
Supuesto	El juego se usa en una red local o de aula, con decenas de jugadores conectados a la vez y no miles.	Con miles de jugadores, un único servidor con estado en memoria (cola, partidas, espectadores) no bastaría y habría que repartir el estado.
Supuesto	La conexión entre cliente y servidor no está expuesta a un atacante con acceso a la red; el atacante esperado es un jugador con un cliente modificado.	Si la red no es de confianza, SecurityMode.None deja expuestas las credenciales y habría que añadir seguridad de transporte (R-02).
Objetivo de negocio	Que las partidas clasificatorias sean justas: el ranking solo vale si nadie puede hacer trampa ni perder elo por una falla del sistema.	Si el ranking no importara, la validación en el servidor seguiría siendo necesaria para las reglas, pero las transacciones del cierre serían menos críticas.
Objetivo de negocio	Que la economía de monedas y cajas sea confiable y transparente: probabilidades visibles y sin dinero real (CU-38 RN-01, CU-39 RN-01).	Introducir dinero real traería obligaciones legales y de seguridad fuera del alcance del curso.
Condición organizacional	Equipo de dos personas, una en macOS arm64 y otra en Windows, durante el periodo agosto 2026 – enero 2027, con entregas parciales evaluadas.	Con menos tiempo se recortarían funciones de prioridad baja (cajas, espectadores), no la estructura.
Condición organizacional	El repositorio remoto se alojará en GitLab o Bitbucket, según lo que pida el curso (decisión abierta, R-07).	No afecta la arquitectura; solo el flujo de trabajo.

2.5 Escenarios de calidad y árbol de utilidad

Entre 8 y 12 escenarios medibles; copien la ficha para cada uno. La respuesta dice qué hace el sistema y la medida dice cómo se comprueba. No escriban la solución dentro del escenario.

QAS-01 · Seguridad · Jugada ilegal desde un cliente modificado	
Fuente	Un jugador con un cliente modificado o con una herramienta propia que habla net.tcp
Estímulo	Envía una jugada ilegal: mover el peón dos casillas sin salto, colocar un muro que encierra al rival, jugar fuera de turno o repetir el número de orden de una jugada ya aplicada
Entorno	Partida clasificatoria en curso, operación normal
Artefacto	Operaciones de partida del servidor (CU-20, CU-21)
Respuesta	El servidor recalcula la jugada sobre su propio estado, la rechaza con un resultado codificado, no cambia el tablero ni el turno y registra el intento
Medida	El 100 % de los casos de una batería de al menos 30 jugadas ilegales (una por regla de CU-20 y CU-21) se rechaza y el estado guardado queda idéntico; se comprueba con pruebas unitarias de los servicios contra dobles de los repositorios
Prioridad	(A, A)
Activo que se protege	El estado de la partida y, por él, el elo y las monedas que otorga
Actor de amenaza	Jugador autenticado que controla su propio cliente
Límite de confianza	La conexión net.tcp entre el cliente y el servidor: nada de lo que llega del cliente se considera válido sin verificarlo

QAS-02 · Seguridad · Adivinación de contraseñas	
Fuente	Un atacante que conoce el nickname o el correo de una víctima
Estímulo	Prueba contraseñas en serie contra el inicio de sesión
Entorno	Operación normal; el canal no tiene seguridad de transporte
Artefacto	Servicio de cuentas y tabla de cuentas
Respuesta	Responde siempre con el mismo resultado genérico, bloquea la cuenta de forma escalonada y registra cada intento en la bitácora de accesos sin la contraseña recibida
Medida	Al tercer fallo consecutivo la cuenta queda bloqueada 5 minutos, al sexto 30 y desde el noveno 60 (CU-01 RN-03); en 100 intentos, la diferencia entre el tiempo medio de respuesta con cuenta inexistente y con contraseña incorrecta es menor a 50 ms; ninguna fila de la base ni línea de bitácora contiene una contraseña en claro. Se comprueba con pruebas unitarias del servicio y una medición sobre el servidor local
Prioridad	(A, M)
Activo que se protege	Las credenciales de la cuenta y todo lo que posee: elo, monedas y objetos
Actor de amenaza	Persona sin la contraseña de la víctima, con un cliente propio y acceso a la red del servidor
Límite de confianza	La operación de inicio de sesión del servidor y, detrás de él, la base de datos, a la que solo llega el servidor con un usuario de mínimo privilegio

QAS-03 · Desempeño · Jugada vista por el rival

Fuente	Un jugador
Estímulo	Confirma una jugada válida en su turno
Entorno	Red local del aula, con 20 partidas simultáneas en el servidor
Artefacto	Servicio de partida, base de datos y cliente del rival
Respuesta	El servidor valida y guarda la jugada y la notifica a todos los participantes
Medida	El percentil 95 del tiempo entre que el servidor recibe la jugada y el cliente del rival la recibe es menor a 500 ms; se mide con marcas de tiempo en las bitácoras de servidor y cliente en una prueba de carga con clientes simulados
Prioridad	(A, A)

QAS-04 · Desempeño · Inicio de sesión

Fuente	Un jugador registrado
Estímulo	Inicia sesión con credenciales correctas
Entorno	Operación normal, sin segundo factor activado
Artefacto	Servicio de cuentas
Respuesta	Valida las credenciales, abre la sesión y entrega el token
Medida	Menos de 1 segundo desde que el jugador pulsa «Iniciar sesión» hasta que ve el menú, sin contar servicios externos (CU-01 RN-11), en el 95 % de 50 intentos; se mide con cronómetro en el cliente
Prioridad	(M, M)

QAS-05 · Disponibilidad · Desconexión de un jugador

Fuente	La red del jugador o el cierre inesperado de su cliente
Estímulo	El jugador pierde la conexión a mitad de una partida
Entorno	Partida en curso, el servidor sigue funcionando
Artefacto	Partida en curso y cliente del jugador
Respuesta	La partida continúa con el reloj corriendo; el rival puede reclamar la victoria a los 60 segundos; si el jugador vuelve, recibe el estado completo y sigue jugando (CU-24, CU-26)
Medida	Ninguna jugada guardada se pierde; tras reconectar, el tablero del cliente coincide con el del servidor en menos de 2 segundos; se comprueba desconectando el cable o cerrando el cliente en 10 partidas de prueba
Prioridad	(A, A)

QAS-06 · Confiabilidad · Falla de la base de datos al cerrar una partida

Fuente	La base de datos
Estímulo	Deja de responder mientras se escriben los resultados de una partida terminada
Entorno	Cierre de una partida clasificatoria (CU-25)
Artefacto	Servicio de partida y repositorios de partidas, estadísticas y monedas
Respuesta	No queda escrito ningún resultado parcial; la partida se marca pendiente de cierre y se reintenta (CU-25 RN-12, RN-13)
Medida	Cero partidas con elo aplicado y monedas sin aplicar (o al revés) en 20 fallas provocadas; la partida se cierra en menos de 1 minuto después de que la base vuelve; se comprueba deteniendo el contenedor de SQL Server durante el cierre
Prioridad	(A, M)

QAS-07 · Integridad · Compra con respuesta perdida

Fuente	Un jugador
Estímulo	Confirma la compra de un objeto o de una caja y la respuesta no llega a su cliente
Entorno	Operación normal con pérdida de conexión justo después de enviar
Artefacto	Servicio de tienda y movimientos de monedas
Respuesta	La compra se aplica una sola vez o ninguna; el cliente consulta el estado en lugar de reenviar (CU-38 RN-08, RN-09)
Medida	En 100 compras con la conexión cortada, el saldo de cada cuenta es igual a la suma de sus movimientos y ninguna cuenta tiene un objeto cobrado dos veces; se comprueba con una consulta SQL sobre la base después de la prueba
Prioridad	(A, M)

QAS-08 · Portabilidad · Dos sistemas operativos

Fuente	Un integrante del equipo
Estímulo	Clona el repositorio en macOS arm64 o en Windows, compila y ejecuta
Entorno	Desarrollo, con el SDK de .NET 10 y SQL Server 2022 en Docker
Artefacto	Toda la solución
Respuesta	Compila, pasa las pruebas y el cliente de un sistema juega una partida completa contra un servidor en el otro
Medida	Cero cambios de código o de configuración del repositorio entre sistemas; se comprueba antes de cada entrega con una partida cruzada Mac–Windows en ambos sentidos
Prioridad	(A, M)

QAS-09 · Modificabilidad · Cambio del proveedor de correo

Fuente	El equipo
Estímulo	Necesita cambiar el servicio de correo o el mecanismo del segundo factor (CU-01 RN-08)
Entorno	Desarrollo
Artefacto	Servicios de cuentas
Respuesta	Se reemplaza la implementación sin tocar los flujos de los casos de uso
Medida	El cambio modifica solo una clase de implementación y su registro en la raíz de composición; cero cambios en contratos, en el cliente y en los servicios que la usan; se comprueba con el diff del cambio
Prioridad	(B, B)

QAS-10 · Capacidad de prueba · Reglas sin infraestructura

Fuente	Un integrante del equipo
Estímulo	Ejecuta las pruebas de las reglas del juego y de los servicios
Entorno	Desarrollo, sin base de datos ni servidor en ejecución
Artefacto	Reglas del juego y servicios del servidor
Respuesta	Las pruebas corren contra el dominio y contra dobles de los repositorios
Medida	El 100 % de las reglas de negocio de CU-20 y CU-21 tiene al menos una prueba; la suite unitaria completa termina en menos de 30 segundos en cualquiera de las dos máquinas
Prioridad	(M, B)

QAS-11 · Usabilidad · Cambio de idioma

Fuente	Un jugador
Estímulo	Cambia el idioma de en-US a es-MX
Entorno	Operación normal
Artefacto	Cliente y resultados del servidor
Respuesta	Todas las pantallas y todos los mensajes de error se muestran en el idioma elegido; el chat no se traduce (CU-28 RN-12)
Medida	Cero textos visibles fuera de los recursos .resx; cada resultado codificado del servidor tiene clave en ambos idiomas; se comprueba con una prueba que compara las claves de los dos archivos y las de las enumeraciones de resultado
Prioridad	(M, B)

QAS-12 · Escalabilidad · Espectadores de una partida	
Fuente	Hasta 200 espectadores (CU-34 RN-03)
Estímulo	Se conectan a la misma partida
Entorno	Partida clasificatoria en curso
Artefacto	Servicio de espectadores y servicio de partida
Respuesta	Cada espectador recibe las jugadas con el retraso fijo; los jugadores no notan la carga
Medida	Retraso de 8 ± 1 segundos para todos los espectadores; el percentil 95 de QAS-03 para los jugadores no empeora más de 10 %; se mide con 200 clientes simulados
Prioridad	(M, A)

Escenario de seguridad. Al menos uno. Agreguen a su ficha tres filas: activo que se protege, actor de amenaza y límite de confianza.

Árbol de utilidad: insértenlo aquí o usen una tabla equivalente. Prioricen por importancia y dificultad.

Utilidad de Bastión			
Atributo	Refinamiento	Escenario	(Imp., Dif.)
Seguridad	Integridad de la partida frente a un cliente modificado	QAS-01	(A, A)
	Confidencialidad de las credenciales sin seguridad de transporte	QAS-02	(A, M)
Desempeño	Latencia de la jugada hacia el rival	QAS-03	(A, A)
	Tiempo de inicio de sesión	QAS-04	(M, M)
	Difusión a espectadores sin afectar a los jugadores	QAS-12	(M, A)
Disponibilidad	Continuidad de la partida ante desconexiones	QAS-05	(A, A)
Confiabilidad e integridad	Atomicidad del cierre de la partida	QAS-06	(A, M)
	Economía sin cobros dobles	QAS-07	(A, M)
Portabilidad	Mismo código en macOS arm64 y Windows	QAS-08	(A, M)
Mantenibilidad	Sustituir servicios externos	QAS-09	(B, B)
	Probar reglas sin infraestructura	QAS-10	(M, B)
Usabilidad	Idiomas en-US y es-MX	QAS-11	(M, B)

A = alta, M = media, B = baja. Los escenarios (A, A) —QAS-01, QAS-03 y QAS-05— son los que más condicionan la arquitectura.

2.6 ASR prioritarios

Entre 5 y 7. Si todos vienen de escenarios, expliquen en una línea por qué ninguna función, restricción, objetivo o condición del equipo obliga a estructurar el sistema.

ASR	Qué debe cumplir la arquitectura	Procedencia	Viene de
ASR-01	El servidor es la única autoridad: toda regla del juego, de la cuenta y de la economía se valida y se decide en el servidor; el cliente solo envía intenciones y muestra el estado que recibe.	Escenario, Función	QAS-01, QAS-07, CU-20, CU-21, CU-38, CU-39
ASR-02	El servidor debe poder avisar a los clientes sin que pregunten (jugadas, chat, emparejamiento, espectadores), con llamadas asíncronas y usando solo tecnologías permitidas por el curso.	Escenario, Restricción	QAS-03, QAS-12, CU-17, CU-28, CU-34, restricción de tecnologías del curso
ASR-03	Cliente y servidor deben compilarse y ejecutarse sin cambios en macOS arm64 y en Windows.	Restricción, Condición organizacional	QAS-08, restricción multiplataforma, equipo Mac-Windows
ASR-04	Sin seguridad de transporte, la seguridad descansa en el servidor: contraseñas con resumen y sal, token de sesión en cada llamada, bloqueo escalonado, bitácoras de solo inserción y acceso a la base con mínimo privilegio.	Escenario, Supuesto	QAS-02, CU-01, CU-44, CU-48, supuesto de red de confianza
ASR-05	Toda operación que cambia una partida o la economía se guarda de forma atómica antes de notificarse, para que una falla del cliente, de la red o de la base no deje estados parciales.	Escenario, Función	QAS-05, QAS-06, QAS-07, CU-25, CU-26
ASR-06	Las reglas del juego viven aisladas, sin dependencias, y los servicios acceden a los datos a través de interfaces, para probarlas sin infraestructura y para que la IA las reutilice.	Condición organizacional, Escenario, Función	QAS-09, QAS-10, CU-27, equipo de dos personas
ASR-07	El servidor devuelve resultados codificados, nunca textos; todo texto visible está en recursos por idioma y el texto se guarda en Unicode.	Función, Escenario	QAS-11, CU-01 RN-12, CU-28 RN-12, D-21

3. Decisiones

Primera entrega · 4 a 6 ADR

Una ficha por decisión esencial. Cada decisión compara al menos dos alternativas reales. Una decisión que cambia no se borra: se marca como sustituida y la nueva la menciona.

3.1 Índice de decisiones

ADR	Decisión	Estado	ASR que atiende
ADR-001	Organizar el sistema en capas cliente-servidor con el servidor como única autoridad	Aceptado	ASR-01, ASR-05, ASR-06
ADR-002	Comunicar cliente y servidor con CoreWCF dúplex sobre net.tcp y llamadas asíncronas	Aceptado	ASR-02, ASR-03
ADR-003	Construir el cliente con MonoGame DesktopGL	Aceptado	ASR-03
ADR-004	Usar SecurityMode.None y concentrar la seguridad en el servidor	Aceptado	ASR-04, ASR-03
ADR-005	Persistir con EF Core y SQL Server 2022, con el esquema en scripts SQL	Aceptado (confirmar con el profesor, R-01)	ASR-04, ASR-05, ASR-03
ADR-006	Traducir en el cliente los resultados codificados del servidor	Aceptado	ASR-07

3.2 Fichas de decisión

ADR-001 · Organizar el sistema en capas cliente-servidor con el servidor como única autoridad	
Estado	Aceptado
Contexto	Un cliente modificado no debe poder hacer trampa (QAS-01), la economía no admite cobros dobles (QAS-07), el cierre de partida debe ser atómico (QAS-06) y el equipo necesita probar las reglas sin infraestructura (QAS-10). Están en juego ASR-01, ASR-05 y ASR-06.
Alternativas	A. Cliente-servidor en capas con servidor autoritativo: el cliente envía intenciones; el servidor valida con un dominio sin dependencias y guarda a través de repositorios. A favor: un solo lugar decide; el dominio se prueba solo y la IA lo reutiliza; solo el servidor toca la base. En contra: cada jugada cuesta un viaje de ida y vuelta; el servidor es punto único de falla (R-05). B. Igual a igual, con el cliente de un jugador como anfitrión de la partida. A favor: sin servidor para jugar; menor latencia para el anfitrión. En contra: el anfitrión puede hacer trampa, lo que invalida el ranking; el elo y las monedas igual necesitan un servidor central. C. Reglas en el cliente y servidor como simple relé y almacén. A favor: respuesta inmediata; servidor más sencillo. En contra: un cliente modificado decide el resultado; contradice CU-20 RN-01 y CU-38 RN-03.
Criterios	Integridad frente a clientes modificados (ASR-01), atomicidad de lo que se guarda (ASR-05), capacidad de prueba y reutilización de reglas (ASR-06), esfuerzo para dos personas.
Decisión	A. Patrón cliente-servidor con arquitectura en capas: cliente (presentación) → contratos; host → servicios → dominio; servicios → acceso a datos → base de datos. Patrón repositorio (IXRepository) entre servicios y datos. El cliente recibe el estado por callbacks y lo calcula solo para dar respuesta inmediata, nunca para decidir.
Consecuencias	Se gana un único punto de verdad, pruebas del dominio y de los servicios con dobles, y reglas de dependencia comprobables. Se paga la latencia de ida y vuelta en cada jugada (QAS-03) y la dependencia de un único servidor.
Riesgos	R-05, R-06
Elementos afectados	C-01, C-02, C-03, C-04, C-05, C-06

ADR-002 · Comunicar cliente y servidor con CoreWCF dúplex sobre net.tcp y llamadas asíncronas	
Estado	Aceptado
Contexto	El curso pide comunicación dúplex sobre net.tcp con llamadas asíncronas y prohíbe stack web, WebSockets, SignalR, gRPC y REST. El servidor debe avisar sin que se le pregunte (jugadas, chat, emparejamiento, espectadores: QAS-03, QAS-12) y el servidor debe correr en macOS y en Windows (QAS-08). Están en juego ASR-02 y ASR-03.
Alternativas	A. CoreWCF 1.9 sobre .NET 10, NetTcpBinding dúplex, contratos en una biblioteca netstandard2.0 y cliente con System.ServiceModel. A favor: corre en ambos sistemas; callbacks en el mismo canal; contratos compartidos. En contra: CoreWCF no implementa todo WCF; usa reflexión, por lo que no se puede recortar ni compilar AOT. B. WCF clásico sobre .NET Framework 4.8. A favor: es lo que se ve en clase; implementación completa. En contra: solo Windows, de modo que el servidor viviría en una sola máquina del equipo (punto único de falla para el desarrollo) y el integrante de macOS no podría ejecutarlo. C. CoreWCF de petición-respuesta con sondeo periódico del cliente. A favor: contratos más simples, sin callbacks. En contra: la latencia depende del intervalo de sondeo (incumple QAS-03 si es largo, sobrecarga el servidor si es corto); 200 espectadores sondeando multiplican la carga (QAS-12).
Criterios	Tecnologías permitidas, notificación del servidor al cliente (ASR-02), multiplataforma (ASR-03), latencia y carga.
Decisión	A. Contratos de servicio y de callback en <code>Bastion.Contracts</code> (netstandard2.0), operaciones asíncronas en el cliente y difusión por callbacks en el servidor. <code>PublishTrimmed</code> y <code>PublishAot</code> quedan desactivados en <code>Directory.Build.props</code> .
Consecuencias	Se gana notificación inmediata y un solo canal por cliente. Se paga un canal con estado que hay que vigilar (fallas del canal = desconexión, CU-26).
Riesgos	R-06, R-09
Elementos afectados	C-01, C-02, C-03, C-04

ADR-003 · Construir el cliente con MonoGame DesktopGL	
Estado	Aceptado
Contexto	El cliente es un juego de escritorio que debe correr en macOS arm64 y en Windows (ASR-03, QAS-08), dibujar el tablero y las pantallas, y ser código C# que el equipo escribe y que se evalúa.
Alternativas	A. MonoGame DesktopGL 3.8.5. A favor: biblioteca de código, corre en macOS y Windows con OpenGL; todo lo hace el código C# del equipo. En contra: no trae controles de interfaz, por lo que el equipo construye los suyos. B. Godot con C#. A favor: editor de escenas y controles de interfaz listos. En contra: buena parte del trabajo se hace en su editor y no en código, que es lo evaluado. C. Unity. A favor: motor completo y documentación amplia. En contra: sobredimensionado para un tablero 2D; proyecto pesado de versionar entre dos personas. D. Stride. A favor: motor C# de código abierto. En contra: su editor solo funciona en Windows. E. Avalonia. A favor: interfaz de escritorio multiplataforma en C#. En contra: no es un motor de juego: sin bucle de juego ni animación del tablero. F. MonoGame WindowsDX. A favor: DirectX nativo. En contra: solo Windows.
Criterios	Multiplataforma (ASR-03), que el trabajo esté en código evaluable, peso del proyecto y curva de aprendizaje.
Decisión	A. El cliente es Bastion.Client con MonoGame DesktopGL, un conjunto propio de controles y pantallas, y un navegador de pantallas.
Consecuencias	Se gana control total y un cliente idéntico en ambos sistemas. Se paga construir controles (campos de texto, listas, ventanas emergentes) y decidir cómo se presenta el tablero (R-03).
Riesgos	R-03
Elementos afectados	C-01

ADR-004 · Usar SecurityMode.None y concentrar la seguridad en el servidor	
Estado	Aceptado
Contexto	La seguridad por omisión de net.tcp usa autenticación de Windows, que no funciona desde macOS (ASR-03). Aun así hay que proteger credenciales y partidas (QAS-01, QAS-02, ASR-04).
Alternativas	A. SecurityMode.None compensado en el servidor: validación de todo, contraseñas PBKDF2-SHA512 con 210 000 iteraciones y sal por cuenta, comparación en tiempo constante, mismo resultado ante cualquier fallo, bloqueo escalonado, token de sesión del que solo se guarda su SHA-256, bitácora de accesos y base accesible solo con un usuario sin DELETE. A favor: funciona entre Mac y Windows; protege lo que se guarda. En contra: lo que viaja por la red va en claro. B. Seguridad de transporte con autenticación de Windows (por omisión). A favor: cifrado sin configuración en Windows. En contra: no funciona desde macOS. C. Seguridad de transporte con certificado TLS. A favor: cifra el canal. En contra: hay que emitir y distribuir certificados a cada cliente y no se ha comprobado que el cliente de .NET en macOS lo soporte con CoreWCF.
Criterios	Funcionar entre ambos sistemas (ASR-03), proteger credenciales y estado (ASR-04), esfuerzo de despliegue.
Decisión	A, con el supuesto de una red de confianza (sección 2.4). La alternativa C queda como mejora si el supuesto resulta falso.
Consecuencias	Se gana un sistema que funciona en ambos sistemas y que no guarda secretos en claro. Se paga que un atacante en la misma red puede leer contraseñas y tokens en tránsito.
Riesgos	R-02
Elementos afectados	C-02, C-03, C-04, C-06, C-07

ADR-005 · Persistir con EF Core y SQL Server 2022, con el esquema en scripts SQL	
Estado	Aceptado (confirmar con el profesor, R-01)
Contexto	Las operaciones de partida y economía deben ser atómicas (ASR-05), la base debe correr en las dos máquinas (ASR-03) y el servidor debe entrar con mínimo privilegio (ASR-04). Los scripts SQL del modelo de datos ya existen en los requisitos.
Alternativas	A. EF Core 10 con el proveedor de SQL Server, sin migraciones: los scripts SQL mandan y el código se ajusta a ellos; SQL Server 2022 en Docker. A favor: transacciones y consultas tipadas; un solo origen del esquema. En contra: hay que mantener a mano la correspondencia entre entidades y tablas. B. EF Core con migraciones. A favor: el esquema se genera desde el código. En contra: dos fuentes de verdad, los scripts y las migraciones, que tarde o temprano divergen. C. ADO.NET con SQL escrito a mano. A favor: control total y lo más cercano a lo visto en clase. En contra: mucho más código repetitivo y propenso a errores de mapeo. D. Azure SQL Edge en Docker para tener imagen ARM en Mac. A favor: nativo en Apple silicon. En contra: Microsoft lo retiró.
Criterios	Atomicidad (ASR-05), multiplataforma (ASR-03), un solo origen del esquema, esfuerzo.
Decisión	A. BastionDbContext con configuraciones por entidad y repositorios en Bastion.Server.Data; el inicio de sesión BastionServerConnection no tiene permiso DELETE; la cadena de conexión vive en user-secrets o en variables de entorno, nunca en el repositorio. En Mac la imagen x64 corre con Rosetta.
Consecuencias	Se gana transacciones con poco código y un esquema único. Se paga la dependencia de Docker y Rosetta en Mac, y que el profesor acepte EF Core.
Riesgos	R-01, R-04
Elementos afectados	C-06, C-07

ADR-006 · Traducir en el cliente los resultados codificados del servidor	
Estado	Aceptado
Contexto	El juego se ofrece en en-US y es-MX (QAS-11); todo texto visible debe venir de catálogos de recursos (CU-01 RN-12, D-21) y el servidor no debe conocer el idioma de cada pantalla (ASR-07).
Alternativas	A. El servidor devuelve enumeraciones de resultado; el cliente las convierte en una clave Pantalla.Elemento y busca el texto en Strings.resx (en-US, neutral y por omisión) o Strings.es-MX.resx. A favor: el contrato no cambia con el idioma; agregar un idioma solo agrega un archivo. En contra: hay que mantener la correspondencia código-clave. B. El servidor genera el mensaje ya traducido según el idioma de la sesión. A favor: el cliente solo lo muestra. En contra: el texto viaja en el contrato; mezclar presentación en el servidor; el cliente no puede adaptar el mensaje a su pantalla. C. Traducciones guardadas en la base de datos. A favor: se cambian sin recompilar. En contra: D-21 lo descarta; más consultas y más tablas sin necesidad.
Criterios	Separación de presentación (ASR-07), contratos estables, esfuerzo para agregar idiomas.
Decisión	A. El idioma por omisión es en-US, lo que sustituye el es-MX por omisión previsto en D-21. Los correos, que salen del servidor, usan los recursos del servidor con el idioma preferido de la cuenta (D-21).
Consecuencias	Se gana un contrato independiente del idioma y una prueba simple de claves faltantes. Se paga mantener dos archivos de recursos sincronizados.
Riesgos	R-08
Elementos afectados	C-01, C-02, C-04

4. Estructura

Primera entrega inicia

Entrega final completa

Describan los elementos y sus reglas con texto y tablas. Los diagramas van en la sección 6 y usan estos mismos identificadores.

4.1 Módulos y componentes

ID	Nombre y tipo	Responsabilidad	Provee / requiere
C-01	Bastion.Client · módulo (ejecutable de escritorio)	Dibuja las pantallas y el tablero, captura la entrada del jugador, muestra los textos en en-US o es-MX y llama a los servicios del servidor; no decide ninguna regla.	Requiere los contratos de servicio de C-02; provee los contratos de callback de C-02.
C-02	Bastion.Contracts · módulo (biblioteca netstandard2.0)	Define los contratos de servicio y de callback, los datos que viajan y los resultados codificados.	Provee IAccountService, ILeaderboardService y los contratos de partida, chat, tienda y moderación; no requiere nada.
C-03	Bastion.Server.Host · componente (proceso del servidor)	Raíz de composición: levanta CoreWCF sobre net.tcp, registra servicios y repositorios en el contenedor de dependencias, lee la configuración y los secretos, y configura log4net.	Provee los extremos net.tcp de C-02; requiere C-04 y C-06.
C-04	Bastion.Server.Services · módulo	Implementa los contratos: valida cada solicitud, aplica la seguridad (resúmenes de contraseña, tokens, bloqueo, auditoría), coordina partidas y transacciones, y difunde por callbacks.	Provee las implementaciones de C-02; requiere C-05, las interfaces IXRepository de C-06 y la interfaz de C-08.
C-05	Bastion.Domain · módulo	Reglas del juego: tablero, peones, muros, saltos, validación de caminos y motor de la IA.	Provee la API de reglas; no requiere nada.
C-06	Bastion.Server.Data · módulo	Acceso a datos con EF Core: contexto, entidades, configuraciones y repositorios.	Provee IAccountRepository, ISessionRepository, IAccessLogRepository, ILeaderboardRepository y IGameModeRepository; requiere C-07.
C-07	Base de datos Bastion · componente (SQL Server 2022 en Docker)	Guarda cuentas, sesiones, partidas, economía y bitácoras; garantiza unicidad y reglas con índices y restricciones.	Provee el esquema de los scripts de database/; se accede solo con BastionServerConnection.
C-08	Pasarela de correo y segundo factor · componente (planeado)	Envía los correos de verificación, recuperación y cambio de correo, y entrega el segundo factor, detrás de una interfaz.	Provee una interfaz de envío a C-04; requiere el servicio de correo externo.

Lo repetitivo se resuelve por convención: escriban la convención una vez, debajo de la tabla.

Convención. Dentro de cada módulo el código se agrupa por funcionalidad (Accounts, Leaderboard, Sessions...) y el espacio de nombres sigue la carpeta; cada tipo va en su propio archivo con su nombre. Cada funcionalidad repite el mismo trío: un contrato IXService en C-02, su implementación XService en C-04 y uno o más IXRepository con su implementación en C-06. Las entidades de EF Core nunca salen de C-04: se convierten en datos del contrato. Las pantallas del cliente siguen el par XScreen y XController, y sus textos usan claves Pantalla.Elemento.

4.2 Interfaces críticas

Sólo las interfaces cuyo incumplimiento produce un riesgo real: dinero, estado compartido, datos personales...

Interfaz	Provista por / usada por	Operaciones	Contrato: condiciones de entrada, resultado garantizado y errores relevantes
[Nombre]	[C-__ / C-__]	[...]	[...]

4.3 Reglas de dependencia

Regla	Permitida o prohibida	Por qué (ADR)
[...]	[...]	[ADR-__]

5. Arquitectura en ejecución Entrega final

Cada preocupación tiene una decisión, o un «no aplica» con su razón. No es una lista de tecnologías: toda tecnología mencionada se conecta con un ADR o un ASR.

Preocupación	Decisión, o «no aplica porque...»	ADR / ASR	Vista o evidencia
Distribución y comunicación	[...]	[...]	[V-__ / EV-__]
Datos, estado y propiedad	[...]	[...]	[V-__ / EV-__]
Concurrencia y coordinación	[...]	[...]	[V-__ / EV-__]
Resiliencia y disponibilidad	[...]	[...]	[V-__ / EV-__]
Seguridad y límites de confianza	[...]	[...]	[V-__ / EV-__]
Despliegue y entornos	[...]	[...]	[V-__ / EV-__]
Observabilidad y operación	[...]	[...]	[V-__ / EV-__]
Costo operacional	[...]	[...]	[V-__ / EV-__]

6. Vistas Primera entrega: V-01 y V-02 Entrega final completa

Una vista existe para responder a una audiencia concreta. Usen la notación que mejor sirva a ese propósito; en el curso se practican UML y C4.

6.1 Tipos de vista que usaremos

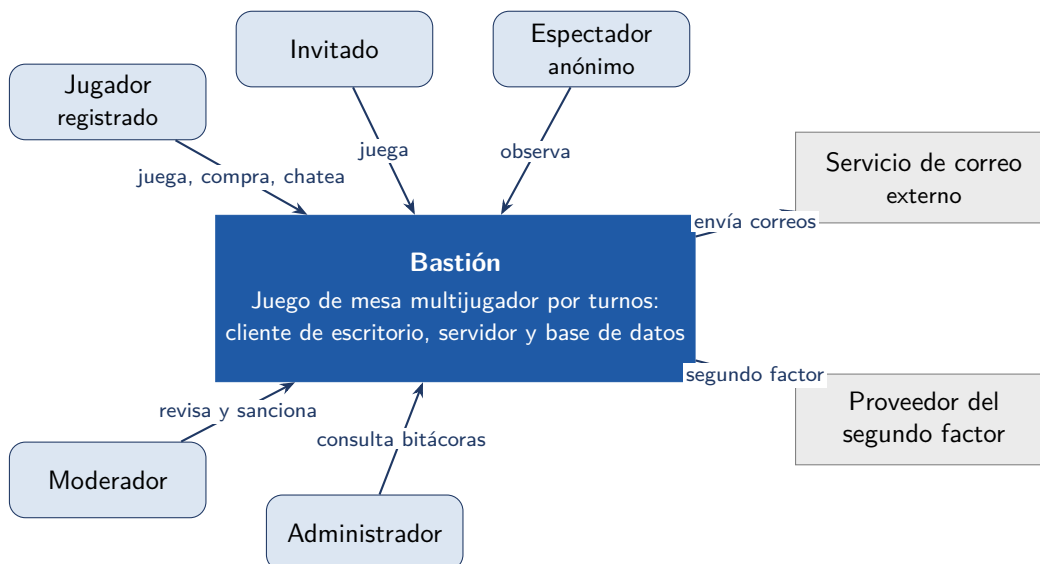
En ISO 42010 estos tipos se llaman viewpoints. No tienen que inventarlos: elijan los que necesiten. Agreguen otro sólo si una preocupación importante queda sin atender.

Tipo de vista	Qué preocupaciones atiende	Representaciones útiles
Contexto	Frontera del sistema, usuarios y sistemas externos	Diagrama de contexto; C4 nivel 1
Estructura	Módulos, componentes, interfaces y dependencias	UML de componentes o de paquetes; C4 niveles 2 y 3
Ejecución	Interacciones, estado compartido, concurrencia y fallas	UML de secuencia, de actividad o máquina de estados
Despliegue	Nodos, comunicación, límites de confianza y redundancia	UML de despliegue

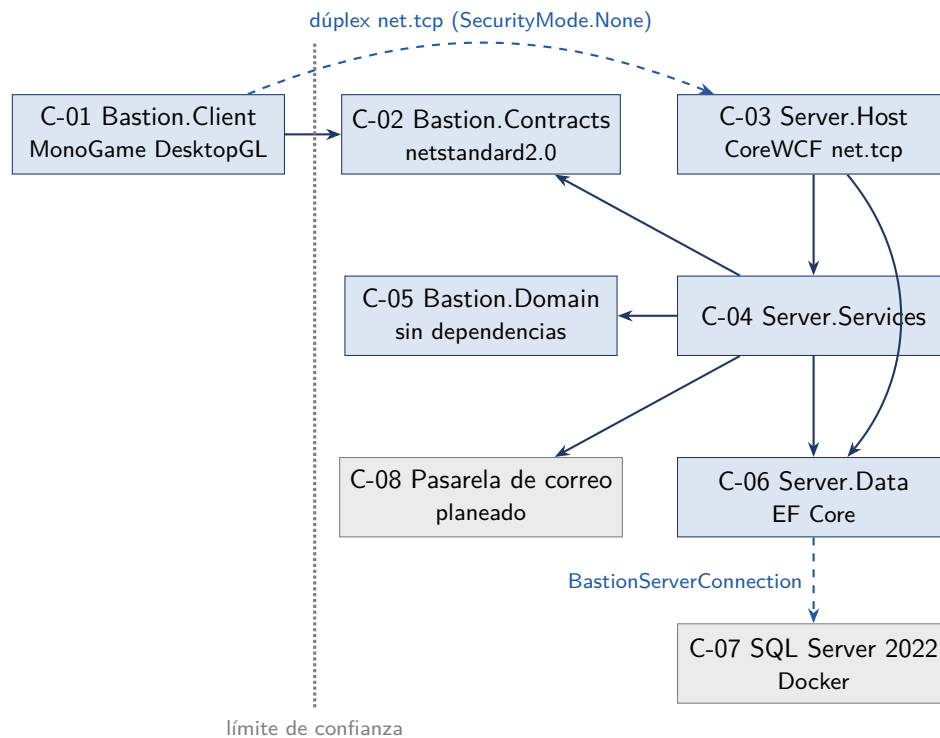
6.2 Vistas

Copien la ficha para cada vista y coloquen el diagrama justo debajo. Declaren al menos una correspondencia con otra vista: qué debe coincidir entre ambas.

V-01 · Contexto de Bastión	
Tipo de vista	Contexto
Para quién y para qué	Para el profesor y el equipo: ver de un vistazo qué está dentro del sistema, quién lo usa y de qué sistemas externos depende.
Preocupaciones que atiende	Frontera del sistema, actores humanos y dependencias externas (sección 2.1).
Notación y leyenda	C4 nivel 1. Figuras: personas; recuadro azul: el sistema; recuadros grises: sistemas externos; flechas: quién inicia la comunicación.
Se corresponde con	V-02: el recuadro de Bastión se descompone en C-01 a C-08, y las flechas hacia el correo y el segundo factor salen de C-08.



V-02 · Estructura en capas	
Tipo de vista	Estructura
Para quién y para qué	Para el equipo: saber en qué módulo va cada clase y qué referencias están permitidas; para el profesor: comprobar que la estructura sigue ADR-001.
Preocupaciones que atiende	Módulos, dependencias permitidas, frontera de red y quién accede a la base de datos.
Notación y leyenda	UML de componentes simplificado. Recuadros: módulos o componentes con su C-xx; flecha continua: dependencia de compilación; flecha discontinua: comunicación por red o por conexión a la base; línea punteada: límite de confianza entre cliente y servidor.
Se corresponde con	V-01: C-01 a C-08 forman el recuadro de Bastión y C-08 es la única salida hacia el correo y el segundo factor; la tabla 4.1: cada recuadro es una fila con el mismo identificador.



7. Evaluación, trazabilidad y transición

Primera entrega: sólo 7.3

Entrega final

Muestren con evidencia qué tan bien responde la arquitectura a sus ASR, qué queda pendiente y qué necesita saber quien la construya.

7.1 Evaluación y trazabilidad de los ASR Entrega final

Una fila por ASR. Esta tabla es la matriz de trazabilidad: no la repitan en otro lugar. En «Resultado» no escriban sólo «cumple»: digan qué mostró la evidencia.

ASR	ADR	Elementos	Vista	Evidencia	Resultado	Riesgo residual
ASR-01	[ADR-__]	[C-__]	[V-__]	[EV-__]	[Qué mostró la evidencia]	[R-__]

7.2 Evidencias Entrega final

Registren aquí cada EV que citen. Una evidencia puede ser una revisión por escenario, una medición, una prueba o un análisis. Es obligatorio incluir al menos una verificación empírica ejecutada y una comprobación de conformidad.

EV	Tipo	Qué se hizo y qué resultó	Archivo o referencia
EV-01	[Revisión / Medición / Prueba / Análisis]	[...]	[evidencia/...]

Verificación empírica · EV-__

Afirmación que se pone a prueba	[Qué esperan de la arquitectura]
Cómo se probó	[Prototipo pequeño (spike), medición o prueba; condiciones]
Resultado	[Qué se observó, con números]
Qué cambia en la arquitectura	[Se confirma, se matiza o se refuta; ajuste realizado]

No necesitan construir el sistema completo: un prototipo pequeño basta si pone a prueba la afirmación.

Comprobación de conformidad · EV-__

Decisión que protege	[ADR-__ o regla de dependencia]
Procedimiento	[Pasos que otra persona puede repetir. Automatizarlo es deseable, no obligatorio]
Resultado esperado	[Cuándo se considera que la regla se violó]

7.3 Riesgos y pendientes Primera entrega inicia

Se inicia en la primera entrega y se mantiene hasta la entrega final.

R	Tipo	Descripción	Qué se hará
R-01	Decisión abierta	El profesor no ha confirmado que EF Core sea aceptable como framework de persistencia en lugar del Entity Framework visto en clase (ADR-005).	Pedir la confirmación por escrito antes de la segunda entrega; si se rechaza, sustituir la implementación de los repositorios de C-06 sin tocar los servicios de C-04.
R-02	Riesgo	Con <code>SecurityMode.None</code> , un atacante en la misma red puede leer contraseñas y tokens en tránsito (ADR-004, QAS-02).	Documentar el supuesto de red de confianza; investigar seguridad de transporte con certificado entre macOS y Windows con un prototipo pequeño.
R-03	Decisión abierta	Presentación del tablero: plano, isométrico 2D o 3D (ADR-003).	Decidir con un prototipo de cada opción en MonoGame antes de construir la pantalla de partida; se registrará como ADR.
R-04	Riesgo	En macOS arm64 no hay imagen ARM de SQL Server 2022; la imagen x64 corre con Rosetta, con posible pérdida de rendimiento o fallas (ADR-005).	Medir el tiempo de las consultas de QAS-04 en Mac; si falla, usar el servidor de base de la máquina Windows durante el desarrollo.
R-05	Riesgo	Un único servidor guarda en memoria la cola, las partidas en curso y los espectadores: si se cae, ese estado se pierde y el servicio se interrumpe (ADR-001, QAS-05).	Escribir cada jugada antes de notificarla (ASR-05) para reconstruir partidas desde la base; aceptar el punto único de falla dado el supuesto de uso en aula.
R-06	Riesgo	No se ha medido la latencia de los callbacks con 20 partidas ni la difusión a 200 espectadores (QAS-03, QAS-12).	Construir un prototipo de carga con clientes simulados para la entrega final (será una EV).
R-07	Decisión abierta	El repositorio remoto será GitLab o Bitbucket.	Elegir según lo que indique el curso; no afecta la arquitectura.
R-08	Pendiente	No se ha elegido el servicio de correo ni el proveedor del segundo factor, y los correos aún no tienen recursos por idioma (C-08, ADR-006).	Definir la interfaz en C-04 y una implementación de prueba que escriba en la bitácora; elegir el proveedor antes de construir CU-02 a CU-04.
R-09	Deuda	CoreWCF depende de reflexión y arrastra dependencias con vulnerabilidades conocidas: una se fijó a mano (<code>System.Security.Cryptography.Xml</code>) y nunca debe activarse <code>PublishTrimmed</code> ni <code>PublishAot</code> (ADR-002).	Revisar los avisos de paquetes en cada entrega; mantener la nota en <code>Directory.Build.props</code> .

7.4 Transición a Construcción Entrega final

Instrucciones accionables para quien implemente: decisiones que no deben romperse, interfaces críticas, comprobaciones que deben ejecutarse y orden sugerido de trabajo. Si ya hay código, registren las diferencias con el diseño; si no, cómo se comprobarán.

[Guía breve.]

Elemento o ADR	Qué dice el diseño	Qué hay en el código, o cómo se comprobará	Qué se hará
[ADR-__ / C-__]	[...]	[...]	[...]

Anexos

Opcionales y sólo para material extenso. Una decisión nueva no puede aparecer únicamente en un anexo.

Lista de comprobación · Primera entrega

- ☐ Contexto, interesados con sus preocupaciones y funciones clave identificados.
- ☐ Restricciones, supuestos, objetivos y condiciones separados por tipo.
- ☐ Entre 8 y 12 escenarios medibles; al menos uno de seguridad con activo, actor de amenaza y límite de confianza.
- ☐ Árbol de utilidad priorizado por importancia y dificultad.
- ☐ Entre 5 y 7 ASR, cada uno con su procedencia.
- ☐ Entre 4 y 6 ADR con estado; cada uno compara al menos dos alternativas reales.
- ☐ Elementos principales en 4.1 y vistas V-01 (contexto) y V-02 (estructura).
- ☐ Riesgos y pendientes iniciados en 7.3; declaración de IA actualizada.

Lista de comprobación · Entrega final

- ☐ Las siete secciones están presentes y no hay relleno sin propósito.
- ☐ Interfaces críticas con contrato y reglas de dependencia declaradas.
- ☐ Cada preocupación de ejecución tiene una decisión o un «no aplica» justificado.
- ☐ Cada vista declara audiencia, propósito y al menos una correspondencia.
- ☐ La tabla 7.1 permite seguir cada ASR hasta su evidencia y su riesgo residual.
- ☐ Hay al menos una verificación empírica ejecutada y una comprobación de conformidad repetible.
- ☐ Hay diferencias registradas con el código, o un plan para comprobarlas, y una guía de transición.
- ☐ Ambos integrantes pueden explicar cualquier decisión del documento.

Entrega. El paquete se arma conforme al Manual vigente. El PDF de este documento es la versión que se califica.